

Assignment#1

NAME: MASHAAL ISHTIAQ

ROLL# : P22-9326

SECTION: BCS-4C

CODE:

```
#include<iostream>
#include<string>

using namespace std;

class Node{

public:

string data;
Node *next;
Node *prev;

Node(string name){
data = name;
next = nullptr;
prev = nullptr;
}

};

class Josephus{

public:

Node *head;
int length;

Josephus(){

head = nullptr;
length = 0;
}

~Josephus(){
Node *temp = head;
while (temp->next != head)
{
Node *ptr;
ptr = temp;
temp = temp->next;
delete ptr;
};
}
void Add(string name){
Node *n = new Node(name);
```

```
if (head == nullptr){
cout<<"List is empty"<<endl;
cout<<"Insertion of first name"<<endl;
head = n;
head->prev = head;
head->next = head;

}
else{

Node *tail = head->prev;
tail->next = n;
n->prev = tail;
n->next = head;
head->prev = n;

}
length++;


}

void display(){
if (head == nullptr) {
cout << "Circle is empty" << endl;
return;
}
Node* curr = head;
cout<<"Current persons in the circle:"<<endl;
do {
cout<< curr->data << " ";
curr = curr->next;
} while (curr != head);

cout << endl;
}

void eliminate_clockwise(string name , int val){
Node* curr = head;
// Node *temp = nullptr;
Node *slow = nullptr;
cout<<"__Elimination Process started__"<<endl;

if(head == nullptr){

cout<<"Circle is empty"<<endl;
return;

}
cout<<"Elimination sequence"<<endl;
int j=1;
```

```
while (length > 1) {
for (int i = 1; i <= val; i++)
{
slow = curr;
curr = curr->next;
}

Node* temp = curr;
cout <<j<<" "<< "Eliminated person: " << temp->data << endl;
j++;
slow->next = temp->next;
temp->next->prev = slow;

if (head == temp)
head = temp->next;

if (curr == head) {
cout << "Error: Attempting to access deleted memory" << endl;
return;
}
curr = curr->next;
delete temp;
length--;

}
if(length == 1){
cout<<"The survivor is "<<curr->data<<endl;
}
}

void eliminate_anticlockwise(string name , int val){
Node* curr = head;
// Node *temp = curr;
Node *slow = nullptr;
cout<<"__Elimination Process started__"<<endl;

if(head == nullptr){

cout<<"Circle is empty"<<endl;
return;

}
cout<<"Elimination sequence"<<endl;
int j=1;
while (length > 1) {
for (int i = 1; i <= val; i++)
{
slow = curr;
curr = curr->prev;
}

Node* temp = curr;
```

```
cout <<j<<" "<< "Eliminated person: " << temp->data << endl;
j++;
slow->prev = temp->prev;
temp->prev->next = slow;

if (head == temp)
head = temp->next;

if (curr == head) {
cout << "Error: Attempting to access deleted memory" << endl;
return;
}
curr = curr->prev;
delete temp;
length--;

}
if(length == 1){
cout<<"The survivor is "<<curr->data<<endl;
}
}

};

int main(){
int choice;
string name;
int val;
char direction;
Josephus *j1= new Josephus();
cout<<"____JOSEPHUS PROBLEM_____"<<endl;
while(1){
cout<<"\n1. Add Person"<<endl;
cout<<"2. Start Elimination Process"<<endl;
cout<<"3. Exit"<<endl;
cout<<"\nEnter your choice"<<endl;
cin>>choice;

switch(choice){

case 1:
{
cout<<"Enter the name for the person: "<<endl;
cin>>name;

j1->Add(name);
cout<<"\nPerson "<<name<<" added to the circle."<<endl;
j1->display();

break;
}
```

```
case 2:{
cout<<"Enter the starting point "<<endl;
cin>>name;

cout<<"Enter the jump value "<<endl;
cin>>val;

cout<<"Choose the direction(a. clockwise b.anti-clockwise)"<<endl;
cin>>direction;
if (direction == 'a' || direction == 'A') {
j1->eliminate_clockwise(name, val);
}
else if (direction == 'b' || direction == 'B') {
j1->eliminate_anticlockwise(name, val);
}
else {
cout << "Invalid direction. Please enter 'a' or 'b'." << endl;
}
break;}

case 3:
cout<<"Exiting the program..."<<endl;
delete j1;
return 0; //to exit from main not only the switch
default:
cout << "Invalid choice. Please enter again." << endl;

}
}
}
```

```
 g++ m.cpp -o m && "/home/masha
JOSEPHUS PROBLEM_

1. Add Person
2. Start Elimination Process
3. Exit

Enter your choice
1
Enter the name for the person:
a
List is empty
Insertion of first name

Person a added to the circle.
Current persons in the circle:
a

1. Add Person
2. Start Elimination Process
3. Exit

Enter your choice
1
```

```
Enter the name for the person:
b

Person b added to the circle.
Current persons in the circle:
a  b

1. Add Person
2. Start Elimination Process
3. Exit

Enter your choice
1
Enter the name for the person:
c

Person c added to the circle.
Current persons in the circle:
a  b  c

1. Add Person
2. Start Elimination Process
3. Exit

Enter your choice
```

```
2. Start Elimination Process
3. Exit

Enter your choice
2
Enter the starting point
a
Enter the jump value
2
Choose the direction(a. clockwi
a
Elimination Process started
Elimination sequence
1 Eliminated person: c
2 Eliminated person: a
The survivor is b

1. Add Person
2. Start Elimination Process
3. Exit

Enter your choice
3
Exiting the program...
mashaal@mashaal-Latitude-7480:~
```